```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
respective (possibly overlapping) bounds *)
let init =
  (false, within 0.00 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 1.00) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Sample from a gaussian, then use
   the resulting value to update flags *)
let step = fun ys ->
  update (gaussian(0,1)) ys in

(* Number of samples determined by
   unbounded recursion *)
let cover = fix cover st :=
  if uniform(0,1) < 0.1 then st
  else cover (step st) in

(* Have all intervals been covered at
   least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run entire process on sampled intervals *)
let final = cover init in
  is_covered final
```

Fig. 4. An infinite stochastic process for random interval covering over $[0, 1]$.

The ~~program~~ notation **discrete**$(p_0 : e_0, \ldots, p_{n-1} : e_{n-1})$ on line 6 is the finite-support surface form from Equation (Finite-support elaboration). In particular, the program processed by type inference and covered by the formal calculus is its elaboration to an index-valued **discrete** followed by finite comparisons and conditionals.

The program models the following process: We start by randomly sampling 4 intervals (l. 6 and 13), stored as a list of pairs, where the first component is a flag and the second component is a function that determines whether a given point falls within that interval. We then employ unbounded recursion (l. 36) to repeat the following a potentially unbounded number of times according to a geometric distribution: sample a point y from a gaussian (l. 31), and update the flag of every interval to true whenever y falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

Slice *automatically* ~~produces a *provably equivalent* (cf. §6) discrete program (~~elaborates the surface program and produces the discrete program shown in §A.5~~). Even though this~~ . The finite-support surface elaboration is semantics-preserving by definition. The broader equivalence between this continuous flagship program and its discretization is the intended consequence of §6 , but remains conditional on the open weak simulation obligation. No current theorem covers this program's Gaussian sampling, lists, and recursion. Even though the discrete program still contains unbounded recursion, our back-end automatically determines that its stochastic execution behavior can be modeled as a *finite-state* Markov chain. ~~Our key insight is that exact inference can then be performed via *probabilistic model checking*: we~~ We use Storm [16], which automatically computes

$e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid ()$	constants
$\mid$ **let** $x = e_1$ **in** e_2	let binding
$\mid$ **uniform**(e_1, e_2) $\mid$ **gaussian**(e_1, e_2) $\mid$ **exponential**(e) $\mid \ldots$	continuous distributions
$\mid$ **discrete**$(p_0, \ldots, p_{n-1})$	discrete distribution
$\mid$ **discrete**$(p_0 : e_0, \ldots, p_{n-1} : e_{n-1})$	surface finite-support distribution
$\mid e_1 < e_2 \mid e_1 \leq e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \leq_{\#n} e_2$	comparisons
$\mid e_1 \,\&\&\, e_2 \mid e_1 \mid\mid e_2 \mid \text{not } e$	Boolean operations
$\mid$ **if** e_1 **then** e_2 **else** e_3	conditional
$\mid (e_1, e_2) \mid$ **fst** $e \mid$ **snd** e	pairs
$\mid$ **fun** $x \;\to\; e \mid e_1\ e_2 \mid$ **fix** $f\ x := e$	functions
$\mid$ **observe** $e \mid e_1; e_2$	effects
$\mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match } e \textbf{ with } \mid \text{nil} \to e_{\text{nil}} \mid e_1 :: e_2 \to e_{\text{cons}} \textbf{ end})$	lists
$\mid$ **diverge**	diverge

Fig. 5. ~~Syntax~~ Core and derived surface syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

that the sought-after probability is 0.034474... for that generated model. See §7.1 for details. ~~To the best of our knowledge, our pipeline is the first capable of performing exact inference on suitable programs containing continuous sampling, higher-order functions, and unbounded recursion.~~ In §7.1.2, we extend this program by conditioning *inside the recursion*.

3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct **observe**(e) conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual discrete distributions and any continuous distribution with a tractable cumulative distribution function (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). Adding further distributions is straightforward as long as their CDFs are known; §C lists the ones implemented.

The expression **discrete**$(p_0, \ldots, p_{n-1})$ requires a nonempty probability vector: $n \geq 1$, every $p_i \geq 0$, and $\sum_i p_i = 1$. It returns $k_{\#n}$ with probability p_k.

These are ideal real-valued premises. The implementation separately validates finite binary64 weights. It normalizes compiler-generated weights only within a tight allowance for accumulated CDF arithmetic, and parses source vectors within a documented 10^{-4} rounded-input allowance. These engineering policies do not replace the exact premise above: binary64 evaluation and normalization do not by themselves establish exact preservation of ideal real-valued probabilities, which requires a separate numerical error analysis.

Listings also use the derived surface form **discrete**$(p_0 : e_0, \ldots, p_{n-1} : e_{n-1})$, which samples expression e_i with probability p_i. This form does not extend the formal core. For a fresh variable

it at $\{\bot\}$ gives the failure equation. Evaluating it at T_τ, subtracting from 1, and using $\sum_i p_i = 1$ gives the missing-mass equation. □

These lemmas concern the exact probability vector embedded in the ideal core term. The parser's `Rounded_input` policy accepts and normalizes binary64 source text before constructing that core term. It does not prove that the resulting binary64 vector equals a mathematical vector intended by the author, and it lies outside the ideal theorem above.

3.2 Types

We formalize the types that Slice supports. A *value* set V is a finite set of reals or $\top$. Elements of $\{<x \mid x \in \mathbb{R}\} \cup \{\leq x \mid x \in \mathbb{R}\}$ are called *cuts*. Cut sets induce intervals partitioning the reals as follows: Given a non-empty cut set $B = \{\sim_1 x_1, \ldots, \sim_n x_n\}$ with $x_1 \leq \ldots \leq x_n$, we define the set $\mathrm{Intervals}(B)$ of intervals it induces as expected, e.g., $\mathrm{Intervals}(\{<0.1, \leq 0.5\}) = \{(-\infty, 0.1), [0.1, 0.5], (0.5, \infty)\}$. The strict and non-strict cuts differ only in which side contains the boundary point: $\mathrm{Intervals}(\{<c\}) = \{(-\infty, c), [c, \infty)\}$, whereas $\mathrm{Intervals}(\{\leq c\}) = \{(-\infty, c], (c, \infty)\}$. Thus a boundary generated by $e < c$ places c on the right, while a boundary generated by $e \leq c$ places c on the left. Moreover, we let $\mathrm{Intervals}(\emptyset) = \{(-\infty, \infty)\}$ and $\mathrm{Intervals}(\top) = 2^{\mathbb{R}}$. We denote by $\mathrm{Intervals}(B)_i$ the i-th interval of $\mathrm{Intervals}(B)$, assuming $\mathrm{Intervals}(B)$ is 0-indexed, i.e., $i \in \{0, ..., n\}$. We denote the set of all cut- and value sets, respectively, by $\mathcal{B}$ and $\mathcal{V}$.

Definition 3.3 (Types). For $n \in \mathbb{N}$, $B \in \mathcal{B}$, and $V \in \mathcal{V}$, types τ adhere to the grammar

$$\tau ::= \mathbf{int} \mid \mathbf{bool} \mid \mathbf{unit} \mid \mathbf{fin}(n) \mid \mathbf{float}[B; V] \mid \tau_1 * \tau_2 \mid \tau_1 \to \tau_2 \mid \mathbf{list}(\tau) \,.$$

Standard types cover integers, booleans, ordered pairs, functions, and lists. Finite types $\mathbf{fin}(n)$ contain $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$; the $\#n$-subscript reminds inference backends such as Dice that these values live in a finite domain. The float type $\mathbf{float}[B; V]$ annotates a real-valued expression with its cut set B (an over-approximation of the comparisons it participates in) and value set V (an over-approximation of the values it can take). $B = \top$ means the expression may be compared to arbitrary reals; $B = \emptyset$ means it will never be compared; $V = \top$ indicates an unrestricted result.

3.3 Typing Rules

The typing rules are split into three parts: Rules specific to float types and continuous distributions (Figure 6), standard rules for all other language constructs (Figure 7), and subtyping (Figure 8).

To understand the float typing rules, it is helpful to understand how discretization works. A type $\mathbf{float}[\mathrm{B}; \mathrm{V}]$ is discretized as follows: If $B \neq \top$ is a finite set of cuts, then the type is discretized to a type $\mathbf{fin}(n+1)$ where $n+1 = |\mathrm{Intervals}(B)|$ is the number of intervals induced by B. For instance, if $B = \{<0.5, \leq 0.8\}$, then the type is discretized to $\mathbf{fin}(3)$, where $0_{\#3}$ represents the interval $(-\infty, 0.5)$, $1_{\#3}$ represents the interval $[0.5, 0.8]$, and $2_{\#3}$ represents the interval $(0.8, \infty)$. If $B = \top$, then no discretization is performed, and the type remains $\mathbf{float}$.

To understand the typing rules, keep in mind that the cut sets B need to be sufficiently fine-grained — i.e., contain enough cuts — so that the discretized program has enough information to determine how the original continuous program behaved.

Constants. Rule Float-Const assigns a constant c the type $\mathbf{float}[B; V]$ whenever $\mathrm{has}(V, c)$ holds (i.e., $c \in V$ or $V = \top$). Constants do not constrain the cut set, but recording them in V ensures later comparisons know which values must remain distinguishable.

Continuous distributions. Rule Continuous-1 types a single-argument distribution when its argument e has type $\mathbf{float}[B; V]$ and $\mathrm{recovers}(B, V)$:

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \textbf{fin}(|B|+1) & \text{if } \tau = \textbf{float}[B;V] \text{ and } B \neq \top \\ \textbf{float}[B;V] & \text{if } \tau = \textbf{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

Crucially, the *recoverability* constraints in the CONTINUOUS-rules from Figure 6 ensure that the parameter expressions of the respective sampling expressions can take only finitely many values, and that the cut sets are fine-grained enough to recover the values the parameter expressions evaluate to, as demonstrated in the preceding example.

A similar intuition applies to discretizable comparisons $e_1 < e_2$, where *both* expressions are non-constant but finite-cut. The *answers* constraints in the rule LESS (resp. LESS-EQUAL) ensure that at least one of e_1 and e_2 evaluates to only finitely many values, which enables discretization:

Example 5.3. Consider the following typed expression $(e_1 \leq e_2) : \textbf{bool}$, given by

```
(if discrete(0.5, 0.5) <#2 1#2 then
       uniform(0,10) : float[B; T] else 2.5 : float[B; {2.5}])
<=
(if discrete(0.5, 0.5) <#2 1#2
       then 2.5 : float[B; {2.5}] else 3.5 : float[B; {3.5}])
```

where $B = \{\leq 2.5, \leq 3.5\}$, and where e_1 : `float[B; T]` and e_2 : `float[B; {2.5,3.5}]`. Intuitively, e_1 samples from a mixture of a uniform and a dirac distribution, and e_2 samples from a finite distribution. Why can $e_1 \leq e_2$ be discretized even though neither e_1 nor e_2 is a constant? As suggested by its type, e_2 can take only finitely many values, namely $\{2.5, 3.5\}$. We apply $\mathcal{D}[\![\cdot]\!]$:

```
(if discrete(0.5, 0.5) <#2 1#2 then discrete(0.25, 0.1, 0.65) else 0#3)
<=
(if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3)
```

Notice that the probability of e_1 evaluating to some value in $(-\infty, 2.5]$ indeed *coincides* with the probability of the discretization of e_1 evaluating to $0_{\#3}$. For e_2, on the other hand, the probability of it evaluating to 2.5 (resp. 3.5) *coincides* with the probability of its discretization evaluating to $0_{\#3}$ (resp. $1_{\#3}$). For this reason, we obtain a semantically equivalent discretization of $e_1 \leq e_2$.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

THEOREM 5.4 (DISCRETIZATION IS WELL-TYPED). *Let* $\Gamma \vdash e : \tau$. *We have* $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, *where* $\mathcal{D}[\![\Gamma]\!]$ *transforms all types in* Γ, *i.e.,* $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ *for all* $x \in \mathrm{dom}(\Gamma)$.

6 Operational Semantics and Soundness

~~We now establish the soundness~~ The intended soundness property of our approach ~~in the following sense: For *every*~~ is that for every core expression $e : \textbf{bool}$, possibly including ~~*continuous sampling, higher-order functions, recursion, and conditioning,*~~ continuous sampling, higher-order functions, recursion, and conditioning,

$e : \textbf{bool}$ *and* $\mathcal{D}[\![e : \textbf{bool}]\!] : \textbf{bool}$ *(obtained from Figure 13) are semantically equivalent.*

The weighted finite-support notation used in Figure 4 is assigned the semantics of its elaborated core expression from Equation (Finite-support elaboration). The theorem therefore applies after

elaboration; it does not add a surface-syntax case to the operational semantics or prove a separate correspondence with some independently defined surface semantics. The first step is to make formal sense of semantic equivalence. For that, we ~~will~~ define a monadic operational semantics ~~for our language~~ in §6.1. ~~We will then proceed by proving the above claim. This is non-trivial: discretization~~ The full compiler theorem requires a weak simulation obligation; that obligation remains open. Discretization decisions are driven by non-local, type-propagated information, so ~~we cannot proceed~~ the missing argument cannot be replaced by a local rule-by-rule ~~argument. To prove the claim rigorously, we relate the executions of e and $\mathcal{D}[\![e]\!]$ through a coupling-style logical relation in §6.1.~~ proof.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics has to distinguish between non-termination and observation failure, which we realize as follows: The "missing" probability mass in $\langle e : \tau\rangle$ is the probability that e diverges. The probability of $\bot$ is the probability of encountering an observation failure. With this model, semantic equivalence indeed captures our sought-after correctness claim: If $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!]\colon Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

Let $\tau = \mathbf{bool}$ for the sake of concreteness. Care must be taken since, e.g., sub-expressions of type $\mathbf{float}[B; \{3.5, 4.5\}]$ cannot evaluate to *arbitrary* reals but are restricted to the values $\{3.5, 4.5\}$. We tackle this by extracting from each type-annotated expression a suitable *skeleton*, where float constants are replaced by "holes" (hence the term "skeleton"). For instance, the expression

$$e \quad = \quad (x : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; y : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

gives rise to the skeleton

$$s \quad = \quad (\square : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

and the type annotations tell us that the possible values these holes can be filled with are in $\mathbb{R} \times \{3.5, 4.5\}$. We can thus inherit a measurable space over all well-typed expressions arising from this particular skeleton s from the standard Borel measurable space over $\mathbb{R} \times \{3.5, 4.5\}$. The measurable space over *all* well-typed expressions of type $\mathbf{bool}$ (i.e., over $Expr_{\mathbf{bool}}$) is then obtained from the measurable spaces of *all* skeletons that give rise to expressions of type $\mathbf{bool}$.

Equipped with this family of measurable spaces, we define the small-step functional $[\![\cdot]\!]$ in the expected, type-preserving manner (cf. Tables 1 and 2). For instance, if e_1 is not a value, then[1]

$$[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!] \;=\; \underbrace{[\![e_1 : \tau_1]\!] \gg\!= \lambda g. \begin{cases} \delta_\bot & \text{if } g = \bot \\ \delta_{(\mathbf{let}\ x = g\ \mathbf{in}\ e_2 : \tau_2) : \tau_2} & \text{otherwise.} \end{cases}}_{\text{standard probabilistic monadic bind over suitable Markov kernels}},$$

[1]Put formally, $(\mu \gg\!= f)(A) = \int f(x)(A) \cdot \mu(dx)$, see Definition F.28 for details. δ_e denotes the Dirac distribution of e.

i.e., $[\![e_1 : \tau_1]\!]$ yields the sub-distribution over expressions obtained from executing e_1 for *one step*, and the monadic bind $\gg\!=$ "folds" this distribution into the surrounding **let**-statement. The intuition for the remaining statements is analogous. Crucially, $\gg\!=$ propagates the probability of encountering $\bot$ — an observation failure — through executions. For instance, assume that e_1 from above is given by **observe** (false : **bool**) : **unit**. Since e_1 encounters an observation failure with probability 1, i.e., $[\![\textbf{observe}\ (\text{false} : \textbf{bool}) : \textbf{unit}]\!](\{\bot\}) = 1$, the same holds for the surrounding **let**-statement, i.e., also $[\![(\textbf{let}\ x = e_1 : \tau_1\ \textbf{in}\ e_2 : \tau_2) : \tau_2]\!](\{\bot\}) = 1$. All details — including proofs that all monadic binds are well-behaved since they operate on suitable Markov kernels — are provided in §F.4.

6.1.2 From Small-Step to Big-Step Semantics. We now define $\langle\cdot\rangle$ via $[\![\cdot]\!]$. For that, we first lift the one-step semantics $[\![\cdot]\!]$ to an n-step semantics $\langle\cdot\rangle_n$ in the expected way:

$$\langle e : \tau\rangle_0 = \delta_{e:\tau} \qquad \text{and} \qquad \langle e : \tau\rangle_{n+1} \;=\; \langle e : \tau\rangle_n \gg\!= \lambda e'.\, \begin{cases} \delta_\bot & \text{if } e' = \bot \\ [\![e' : \tau]\!] & \text{otherwise} \end{cases}$$

This construction is well-defined since the functional $[\![\cdot]\!]$ itself gives rise to a suitable Markov kernel (Lemma F.39). We then define for every type τ the functional $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$ as $\langle e : \tau\rangle = \lambda A.\, \lim_{n\to\infty} \langle e : \tau\rangle_n(A)$, which is well-defined by the monotone convergence theorem: the sequence $(\langle e : \tau\rangle_n)_{n\in\mathbb{N}}$ is monotonically increasing for every measurable set A over $Vals_\tau + \bot$ (cf. Lemma F.47) and upper-bounded by 1: executing a program for more and more steps can only increase the probability of terminating in a given set of values or an observation failure.

6.2 Soundness via Probabilistic Couplings

We first state the central theorem, then outline our coupling-style correctness argument.

Theorem 6.1 (Conditional soundness). ~~*Let e : **bool**. Then*~~ *Assume the weak simulation obligation for the source and discretized operational semantics. For every core expression e : **bool** in its stated fragment,* $\langle e : \textbf{bool}\rangle = \langle \mathcal{D}[\![e : \textbf{bool}]\!] : \textbf{bool}\rangle$.

Intuitively, we prove that the two programs can execute in a synchronized manner such that their probabilities of terminating in value v coincide for each $v \in \{\text{true}, \text{false}, \bot\}$.

Towards this end, define for every type τ the set $P_\tau = \{(e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]) \mid e : \tau\}$. We define a *coupled small-step semantics* $[\![\cdot,\cdot]\!]\colon P_\tau \to \mathcal{D}(P_\tau + \bot)$ (cf. Tables 3 and 4), which models the simultaneous execution of e and $\mathcal{D}[\![e : \tau]\!]$. The key property of this coupled semantics is that the semantics of both e and $\mathcal{D}[\![e : \tau]\!]$ can be recovered in a sense we formalize next.

Given $\mu \in \mathcal{D}(P_\tau + \bot)$, we define the *projection of μ onto the i-th component* for $i \in \{1, 2\}$ as

$$\pi_i(\mu) = \lambda A. \int_{x \in P_\tau + \bot} \begin{cases} \delta_{e_i}(A) & x = (e_1, e_2) \\ \delta_\bot(A) & x = \bot \end{cases} \mu(dx)\,,$$

With projections, we recover the semantics of e and $\mathcal{D}[\![e : \tau]\!]$ as follows:

Lemma 6.2. *Let $e : \tau$. We have:*

(1) $\pi_1([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![e : \tau]\!]$.
(2) $\pi_2([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![\mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]$.

Now, using a limit construction analogous to §6.1.2 for obtaining a coupled *big-step* semantics $\langle\cdot,\cdot\rangle\colon P_\tau \to \mathcal{D}(V_\tau + \bot)$, where $V_\tau = \{(v : \tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau \text{ is a value}\}$, we prove a suitable continuity property of projection (cf. Lemma F.51) to conclude that Lemma 6.2 transfers to $\langle\cdot,\cdot\rangle$. Finally, observing that $V_{\textbf{bool}} + \bot = \{(\text{true}, \text{true}), (\text{false}, \text{false}), \bot\}$ yields the desired claim: